

SQL PLACEMENT RADYNESS ASSIGNMENT:

-- DDL Practice Questions

1. Add a column 'manufacturer' to the Medicines table.

ALTER TABLE Medicines

ADD COLUMN manufacturer VARCHAR(255);

The screenshot displays the MySQL Workbench interface. The left sidebar shows the 'SCHEMAS' panel with a tree view containing 'pharmacydb', 'sakila', 'sys', and 'world'. The main editor window shows a SQL file with the following queries:

```
-- Insert Sample Purchases
57 * INSERT INTO Purchases (purchase_id, supplier_id, medicine_id, quantity, purchase_date, batch_number) VALUES
58 (1, 101, 1, 300, '2024-01-05', 'P100'),
59 (2, 102, 3, 100, '2024-01-10', 'S200'),
60 (3, 101, 2, 200, '2024-02-01', 'C300');
61
-- Insert Sample Sales
62 * INSERT INTO Sales (sale_id, medicine_id, quantity, sale_date, total_price) VALUES
63 (1001, 1, 10, '2024-01-15', 50.00),
64 (1002, 1, 20, '2024-02-10', 100.00),
65 (1003, 3, 5, '2024-02-15', 2250.00),
66 (1004, 2, 15, '2024-02-20', 187.50),
67 (1005, 2, 5, '2024-03-25', 62.50),
68 (1006, 5, 20, '2024-04-01', 1600.00);
69
-- DDL Practice Questions
70
71 -- 1. Add a column 'manufacturer' to the Medicines table.
72 * ALTER TABLE Medicines
73 ADD COLUMN manufacturer VARCHAR(255);
74
```

The right sidebar shows the 'SQLAdditions' panel with a message: 'Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.'

The bottom panel shows the 'Output' window with a table of execution results:

#	Time	Action	Message	Duration / Fetch
12	19:41:53	CREATE TABLE Sales (sale_id INT PRIMARY KEY, medicine_id INT, quantity INT, sale_date DATE...	Error Code: 1050. Table 'sales' already exists	0.000 sec
13	19:42:27	INSERT INTO Medicines (medicine_id, name, type, unit_price, stock_quantity, expiry_date) VALUES (1, 'Parac...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.000 sec
14	19:42:27	INSERT INTO Suppliers (supplier_id, name, contact_number, city) VALUES (101, 'Health Corp', '9876543210', ...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.015 sec
15	19:42:27	INSERT INTO Purchases (purchase_id, supplier_id, medicine_id, quantity, purchase_date, batch_number) VAL...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.000 sec
16	19:42:27	INSERT INTO Sales (sale_id, medicine_id, quantity, sale_date, total_price) VALUES (1001, 1, 10, '2024-01-15', ...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.000 sec
17	19:55:41	ALTER TABLE Medicines ADD COLUMN manufacturer VARCHAR(255)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec

2. Rename the column 'contact_number' to 'phone' in Suppliers.

ALTER TABLE Suppliers

RENAME COLUMN contact_number TO phone;

The screenshot shows the MySQL Workbench interface with a local instance of MySQL 8.0. The left sidebar displays the 'SCHEMAS' tree, highlighting the 'pharmacydb' database. The main editor window contains an 'SQL File 4' with the following SQL script:

```
61
62 -- Insert Sample Sales
63 INSERT INTO Sales (sale_id, medicine_id, quantity, sale_date, total_price) VALUES
64 (1001, 1, 10, '2024-01-15', 50.00),
65 (1002, 1, 20, '2024-02-10', 100.00),
66 (1003, 3, 5, '2024-02-15', 2250.00),
67 (1004, 2, 15, '2024-02-20', 187.50),
68 (1005, 2, 5, '2024-03-25', 62.50),
69 (1006, 5, 20, '2024-04-01', 1600.00);
70
71 -- DDL Practice Questions
72 -- 1. Add a column 'manufacturer' to the Medicines table.
73 ALTER TABLE Medicines
74 ADD COLUMN manufacturer VARCHAR(255);
75
76 -- 2. Rename the column 'contact_number' to 'phone' in Suppliers.
77
78 ALTER TABLE Suppliers
79 RENAME COLUMN contact_number TO phone;
80
```

The right sidebar shows a message: "Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help."

The bottom panel displays the 'Output' window, showing the execution results of the SQL script. The results are as follows:

#	Time	Action	Message	Duration / Fetch
13	19:42:27	INSERT INTO Medicines (medicine_id, name, type, unit_price, stock_quantity, expiry_date) VALUES (1, 'Paras...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.000 sec
14	19:42:27	INSERT INTO Suppliers (supplier_id, name, contact_number, city) VALUES (101, 'Health Corp', '9876543210', '...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.015 sec
15	19:42:27	INSERT INTO Purchases (purchase_id, supplier_id, medicine_id, quantity, purchase_date, batch_number) VAL...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.000 sec
16	19:42:27	INSERT INTO Sales (sale_id, medicine_id, quantity, sale_date, total_price) VALUES (1001, 1, 10, '2024-01-15', ...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.000 sec
17	19:55:41	ALTER TABLE Medicines ADD COLUMN manufacturer VARCHAR(255)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
18	19:57:35	ALTER TABLE Suppliers RENAME COLUMN contact_number TO phone	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec

DML Practice Questions

3. Insert a new medicine 'Amoxicillin'. (Done in sample data, but re-run if needed)

INSERT INTO Medicines (medicine_id, name, type, unit_price, stock_quantity, expiry_date)

VALUES (5, 'Dolo 650', 'Tablet', 10.00, 400, '2026-10-31');

The screenshot shows the MySQL Workbench interface with a SQL script in the editor and its execution results in the Output pane.

SQL Script:

```
66 (1003, 3, 5, '2024-02-15', 2250.00),
67 (1004, 2, 15, '2024-02-20', 187.50),
68 (1005, 2, 5, '2024-03-25', 62.50),
69 (1006, 5, 20, '2024-04-01', 1600.00);
70
71 -- DDL Practice Questions
72 -- 1. Add a column 'manufacturer' to the Medicines table.
73 • ALTER TABLE Medicines
74 ADD COLUMN manufacturer VARCHAR(255);
75
76 -- 2. Rename the column 'contact_number' to 'phone' in Suppliers.
77
78 • ALTER TABLE Suppliers
79 RENAME COLUMN contact_number TO phone;
80
81 -- DML Practice Questions
82 -- 3. Insert a new medicine 'Amoxicillin'. (Done in sample data, but re-run if needed)
83 • INSERT INTO Medicines (medicine_id, name, type, unit_price, stock_quantity, expiry_date)
84 VALUES (5, 'Dolo 650', 'Tablet', 10.00, 400, '2026-10-31');
85
```

Output:

#	Time	Action	Message	Duration / Fetch
✓ 15	19:42:27	INSERT INTO Purchases (purchase_id, supplier_id, medicine_id, quantity, purchase_date, batch_number) VAL...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.000 sec
✓ 16	19:42:27	INSERT INTO Sales (sale_id, medicine_id, quantity, sale_date, total_price) VALUES (1001, 1, 10, '2024-01-15', ...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.000 sec
✓ 17	19:55:41	ALTER TABLE Medicines ADD COLUMN manufacturer VARCHAR(255)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
✓ 18	19:57:35	ALTER TABLE Suppliers RENAME COLUMN contact_number TO phone	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec
✗ 19	20:04:27	INSERT INTO Medicines (medicine_id, name, type, unit_price, stock_quantity, expiry_date) VALUES (5, 'Dolo ...	Error Code: 1062 Duplicate entry '5' for key 'Medicines.PRIMARY'	0.016 sec
✓ 20	20:10:45	INSERT INTO Medicines (medicine_id, name, type, unit_price, stock_quantity, expiry_date) VALUES (8, 'Dolo ...	1 row(s) affected	0.016 sec

4. Update all medicine prices by 10%.

UPDATE Medicines

SET unit_price = unit_price * 1.10;

The screenshot shows the MySQL Workbench interface with a local instance of MySQL 8.0. The SQL Editor contains the following queries:

```
-- DDL Practice Questions
-- 1. Add a column 'manufacturer' to the Medicines table.
ALTER TABLE Medicines
ADD COLUMN manufacturer VARCHAR(255);

-- 2. Rename the column 'contact_number' to 'phone' in Suppliers.
ALTER TABLE Suppliers
RENAME COLUMN contact_number TO phone;

-- DML Practice Questions
-- 3. Insert a new medicine 'Amoxicillin'. (Done in sample data, but re-run if needed)
INSERT INTO Medicines (medicine_id, name, type, unit_price, stock_quantity, expiry_date)
VALUES (8, 'Dolo 650', 'Tablet', 10.00, 400, '2026-10-31');

-- 4. Update all medicine prices by 10%.
UPDATE Medicines
SET unit_price = unit_price * 1.10;
SET SQL_SAFE_UPDATES = 0;
```

The Output window shows the execution results:

#	Time	Action	Message	Duration / Fetch
18	19:57:35	ALTER TABLE Suppliers RENAME COLUMN contact_number TO phone	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec
19	20:04:27	INSERT INTO Medicines (medicine_id, name, type, unit_price, stock_quantity, expiry_date) VALUES (5, 'Dolo ...	Error Code: 1062 Duplicate entry '5' for key 'medicines.PRIMARY'	0.016 sec
20	20:10:45	INSERT INTO Medicines (medicine_id, name, type, unit_price, stock_quantity, expiry_date) VALUES (8, 'Dolo ...	1 row(s) affected	0.016 sec
21	20:12:20	UPDATE Medicines SET unit_price = unit_price * 1.10	Error Code: 1175 You are using safe update mode and you tried to update a table without a WHERE that uses ...	0.015 sec
22	20:14:26	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
23	20:15:00	UPDATE Medicines SET unit_price = unit_price * 1.10	7 row(s) affected Rows matched: 7 Changed: 7 Warnings: 0	0.031 sec

5. Delete medicines that are expired. (Assuming current date is 2024-03-15)

DELETE FROM Medicines

WHERE expiry_date < '2024-03-15';

The screenshot shows the MySQL Workbench interface with a SQL script in the 'SQL File 4' editor. The script performs several operations: setting SQL_SAFE_UPDATES to 0, using the PharmacyDB database, and deleting records from the Medicines, Sales, and Purchases tables based on an expiry date of 2024-03-15. The output window at the bottom shows the execution results, including error messages for safe update mode and foreign key constraints.

```
92 • DELETE FROM Medicines
93   WHERE expiry_date < '2024-03-15';
94 • SET SQL_SAFE_UPDATES = 0;
95 • USE PharmacyDB;
96 • SET SQL_SAFE_UPDATES = 0; -- Ensure Safe Mode is off (as previously needed)
97
98 -- 1. Delete dependent records from SALES
99
100 • DELETE FROM Sales
101   WHERE medicine_id IN (SELECT medicine_id FROM Medicines WHERE expiry_date < '2024-03-15');
102
103 -- 2. Delete dependent records from PURCHASES
104
105 • DELETE FROM Purchases
106   WHERE medicine_id IN (SELECT medicine_id FROM Medicines WHERE expiry_date < '2024-03-15');
107
108 -- 5. Delete medicines that are expired. (Assuming current date is 2024-03-15)
109 • DELETE FROM Medicines
110   WHERE expiry_date < '2024-03-15';
111
```

Output:

#	Time	Action	Message	Duration / Fetch
25	20:17:21	DELETE FROM Medicines WHERE expiry_date < 2024-03-15'	Error Code: 1175: You are using safe update mode and you tried to update a table without a WHERE that uses ...	0.000 sec
26	20:18:49	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
27	20:19:01	DELETE FROM Medicines WHERE expiry_date < 2024-03-15'	Error Code: 1451: Cannot delete or update a parent row: a foreign key constraint fails ('pharmacydb`.`purchases'...	0.015 sec
28	20:23:36	DELETE FROM Sales WHERE medicine_id IN (SELECT medicine_id FROM Medicines WHERE expiry_date < ...	1 row(s) affected	0.015 sec
29	20:23:36	DELETE FROM Purchases WHERE medicine_id IN (SELECT medicine_id FROM Medicines WHERE expiry_d...	1 row(s) affected	0.000 sec
30	20:24:28	DELETE FROM Medicines WHERE expiry_date < 2024-03-15'	1 row(s) affected	0.015 sec

Aggregations Practice Questions

6. Find total sales revenue per medicine.

```
SELECT M.name, SUM(S.total_price) AS total_revenue  
  
FROM Sales S  
  
JOIN Medicines M ON S.medicine_id = M.medicine_id  
  
GROUP BY M.name;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Aggregations Practice Questions  
-- 6. Find total sales revenue per medicine.  
SELECT M.name, SUM(S.total_price) AS total_revenue  
FROM Sales S  
JOIN Medicines M ON S.medicine_id = M.medicine_id  
GROUP BY M.name;
```

The Result Grid shows the following data:

name	total_revenue
Paracetamol	150.00
Amoxicillin	250.00
Cough Syrup	1600.00

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
26	20:18:49	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
27	20:19:01	DELETE FROM Medicines WHERE expiry_date < '2024-03-15'	Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails (pharmacydb: 'purchase...	0.015 sec
28	20:23:36	DELETE FROM Sales WHERE medicine_id IN (SELECT medicine_id FROM Medicines WHERE expiry_date ...	1 row(s) affected	0.015 sec
29	20:23:36	DELETE FROM Purchases WHERE medicine_id IN (SELECT medicine_id FROM Medicines WHERE expiry_...	1 row(s) affected	0.000 sec
30	20:24:28	DELETE FROM Medicines WHERE expiry_date < '2024-03-15'	1 row(s) affected	0.015 sec
31	20:26:06	SELECT M.name, SUM(S.total_price) AS total_revenue FROM Sales S JOIN Medicines M ON S.medicine_id = ...	3 row(s) returned	0.000 sec / 0.000 sec

7. Find the supplier who provided the maximum quantity.

SELECT S.name

FROM Purchases P

JOIN Suppliers S ON P.supplier_id = S.supplier_id

GROUP BY S.name

ORDER BY SUM(P.quantity) DESC

LIMIT 1;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- 7. Find the supplier who provided the maximum quantity.
SELECT S.name
FROM Purchases P
JOIN Suppliers S ON P.supplier_id = S.supplier_id
GROUP BY S.name
ORDER BY SUM(P.quantity) DESC
LIMIT 1;
```

The Result Grid shows the following result:

name
Health Corp

The Output tab shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
27	20:19:01	DELETE FROM Medicines WHERE expiry_date < '2024-03-15'	Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails (pharmacydb: 'purchase...	0.015 sec
28	20:23:36	DELETE FROM Sales WHERE medicine_id IN (SELECT medicine_id FROM Medicines WHERE expiry_date ...	1 row(s) affected	0.015 sec
29	20:23:36	DELETE FROM Purchases WHERE medicine_id IN (SELECT medicine_id FROM Medicines WHERE expiry_...	1 row(s) affected	0.000 sec
30	20:24:28	DELETE FROM Medicines WHERE expiry_date < '2024-03-15'	1 row(s) affected	0.015 sec
31	20:26:06	SELECT M.name, SUM(S.total_price) AS total_revenue FROM Sales S JOIN Medicines M ON S.medicine_id = ...	3 row(s) returned	0.000 sec / 0.000 sec
32	20:27:02	SELECT S.name FROM Purchases P JOIN Suppliers S ON P.supplier_id = S.supplier_id GROUP BY S.name ...	1 row(s) returned	0.000 sec / 0.000 sec

8. Total units sold per medicine type.

```
SELECT M.type, SUM(S.quantity) AS total_units_sold  
FROM Sales S  
JOIN Medicines M ON S.medicine_id = M.medicine_id  
GROUP BY M.type;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
125 LIMIT 1;  
126  
127 -- 8. Total units sold per medicine type.  
128 SELECT M.type, SUM(S.quantity) AS total_units_sold  
129 FROM Sales S  
130 JOIN Medicines M ON S.medicine_id = M.medicine_id  
131 GROUP BY M.type;
```

The Results Grid shows the following data:

type	total_units_sold
Tablet	30
Capsule	20
Syrup	20

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
28	20:23:36	DELETE FROM Sales WHERE medicine_id IN (SELECT medicine_id FROM Medicines WHERE expiry_date > '2024-03-15')	1 row(s) affected	0.015 sec
29	20:23:36	DELETE FROM Purchases WHERE medicine_id IN (SELECT medicine_id FROM Medicines WHERE expiry_date > '2024-03-15')	1 row(s) affected	0.000 sec
30	20:24:28	DELETE FROM Medicines WHERE expiry_date < '2024-03-15'	1 row(s) affected	0.015 sec
31	20:26:06	SELECT M.name, SUM(S.total_price) AS total_revenue FROM Sales S JOIN Medicines M ON S.medicine_id = M.medicine_id GROUP BY M.name	3 row(s) returned	0.000 sec / 0.000 sec
32	20:27:02	SELECT S.name FROM Purchases P JOIN Suppliers S ON P.supplier_id = S.supplier_id GROUP BY S.name	1 row(s) returned	0.000 sec / 0.000 sec
33	20:27:49	SELECT M.type, SUM(S.quantity) AS total_units_sold FROM Sales S JOIN Medicines M ON S.medicine_id = M.medicine_id GROUP BY M.type	3 row(s) returned	0.015 sec / 0.000 sec

Subqueries, HAVING, and JOINS

SQL

Subqueries Practice Questions

9. Medicines with stock below average.

SELECT name, stock_quantity

FROM Medicines

WHERE stock_quantity < (

SELECT AVG(stock_quantity) FROM Medicines

);

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- Subqueries Practice Questions
-- 9. Medicines with stock below average.
SELECT name, stock_quantity
FROM Medicines
WHERE stock_quantity < (
    SELECT AVG(stock_quantity) FROM Medicines
);
```

The Results window displays the following data:

name	stock_quantity
Vitamin C	100
Cough Syrup	150
Ibuprofen	200

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
29	20:23:36	DELETE FROM Purchases WHERE medicine_id IN (SELECT medicine_id FROM Medicines WHERE expiry_date < '2024-03-15')	1 row(s) affected	0.000 sec
30	20:24:28	DELETE FROM Medicines WHERE expiry_date < '2024-03-15'	1 row(s) affected	0.015 sec
31	20:26:06	SELECT M.name, SUM(S.total_price) AS total_revenue FROM Sales S JOIN Medicines M ON S.medicine_id = M.medicine_id GROUP BY M.name	3 row(s) returned	0.000 sec / 0.000 sec
32	20:27:02	SELECT S.name FROM Purchases P JOIN Suppliers S ON P.supplier_id = S.supplier_id GROUP BY S.name	1 row(s) returned	0.000 sec / 0.000 sec
33	20:27:49	SELECT M.name, SUM(S.quantity) AS total_units_sold FROM Sales S JOIN Medicines M ON S.medicine_id = M.medicine_id GROUP BY M.name	3 row(s) returned	0.015 sec / 0.000 sec
34	20:28:40	SELECT name, stock_quantity FROM Medicines WHERE stock_quantity < (SELECT AVG(stock_quantity) FROM Medicines)	3 row(s) returned	0.000 sec / 0.000 sec

10. Medicines never sold.

SELECT name

FROM Medicines

WHERE medicine_id NOT IN (

SELECT DISTINCT medicine_id FROM Sales

);

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- 10. Medicines never sold.
SELECT name
FROM Medicines
WHERE medicine_id NOT IN (
  SELECT DISTINCT medicine_id FROM Sales
);
```

The Result Grid shows the following results:

name
Vitamin C
Ibuprofen
Dolo 650

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
30	20:24:28	DELETE FROM Medicines WHERE expiry_date < '2024-03-15'	1 row(s) affected	0.015 sec
31	20:26:06	SELECT M.name, SUM(S total_price) AS total_revenue FROM Sales S JOIN Medicines M ON S.medicine_id = M.medicine_id GROUP BY M.name	3 row(s) returned	0.000 sec / 0.000 sec
32	20:27:02	SELECT S.name FROM Purchases P JOIN Suppliers S ON P.supplier_id = S.supplier_id GROUP BY S.name	1 row(s) returned	0.000 sec / 0.000 sec
33	20:27:49	SELECT M.type, SUM(S quantity) AS total_units_sold FROM Sales S JOIN Medicines M ON S.medicine_id = M.medicine_id GROUP BY M.type	3 row(s) returned	0.015 sec / 0.000 sec
34	20:28:40	SELECT name, stock_quantity FROM Medicines WHERE stock_quantity < (SELECT AVG(stock_quantity) FROM Medicines)	3 row(s) returned	0.000 sec / 0.000 sec
35	20:29:19	SELECT name FROM Medicines WHERE medicine_id NOT IN (SELECT DISTINCT medicine_id FROM Sales)	3 row(s) returned	0.016 sec / 0.000 sec

HAVING Practice Questions

11. Suppliers with total supplied quantity > 500.

SELECT S.name

FROM Purchases P

JOIN Suppliers S ON P.supplier_id = S.supplier_id

GROUP BY S.name

HAVING SUM(P.quantity) > 500;

The screenshot shows the MySQL Workbench interface. The SQL Editor window contains the following query:

```
-- 11. Suppliers with total supplied quantity > 500.
SELECT S.name
FROM Purchases P
JOIN Suppliers S ON P.supplier_id = S.supplier_id
GROUP BY S.name
HAVING SUM(P.quantity) > 500;
```

The Results window shows the output of the query, which is a single row with the name 'name'.

The Output window shows the execution log, including the query execution time and the number of rows returned.

#	Time	Action	Message	Duration / Fetch
31	20:26:06	SELECT M.name, SUM(S.total_price) AS total_revenue FROM Sales S JOIN Medicines M ON S.medicine_id = ...	3 row(s) returned	0.000 sec / 0.000 sec
32	20:27:02	SELECT S.name FROM Purchases P JOIN Suppliers S ON P.supplier_id = S.supplier_id GROUP BY S.name ...	1 row(s) returned	0.000 sec / 0.000 sec
33	20:27:49	SELECT M.type, SUM(S.quantity) AS total_units_sold FROM Sales S JOIN Medicines M ON S.medicine_id = ...	3 row(s) returned	0.015 sec / 0.000 sec
34	20:28:40	SELECT name, stock_quantity FROM Medicines WHERE stock_quantity < (SELECT AVG(stock_quantity) ...	3 row(s) returned	0.000 sec / 0.000 sec
35	20:29:19	SELECT name FROM Medicines WHERE medicine_id NOT IN (SELECT DISTINCT medicine_id FROM Sa ...	3 row(s) returned	0.016 sec / 0.000 sec
36	20:30:08	SELECT S.name FROM Purchases P JOIN Suppliers S ON P.supplier_id = S.supplier_id GROUP BY S.name ...	0 row(s) returned	0.016 sec / 0.000 sec

12. Medicine types where avg. price > ₹100.

SELECT type

FROM Medicines

GROUP BY type

HAVING AVG(unit_price) > 100.00;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
153 HAVING SUM(P.quantity) > 500;
154
155 -- 12. Medicine types where avg. price > ₹100.
156 SELECT type
157 FROM Medicines
158 GROUP BY type
159 HAVING AVG(unit_price) > 100.00;
```

The Results window shows the execution of the query. The output is a table with one column, 'type', and one row, 'type'. The Results window also shows the execution plan for the query, which is a simple table scan.

#	Time	Action	Message	Duration / Fetch
32	20:27:02	SELECT S name FROM Purchases P JOIN Suppliers S ON P.supplier_id = S.supplier_id GROUP BY S name	1 row(s) returned	0.000 sec / 0.000 sec
33	20:27:49	SELECT M type, SUM(S.quantity) AS total_units_sold FROM Sales S JOIN Medicines M ON S.medicine_id = M.medicine_id	3 row(s) returned	0.015 sec / 0.000 sec
34	20:28:40	SELECT name, stock_quantity FROM Medicines WHERE stock_quantity < (SELECT AVG(stock_quantity) FROM Medicines)	3 row(s) returned	0.000 sec / 0.000 sec
35	20:29:19	SELECT name FROM Medicines WHERE medicine_id NOT IN (SELECT DISTINCT medicine_id FROM Sales)	3 row(s) returned	0.016 sec / 0.000 sec
36	20:30:08	SELECT S name FROM Purchases P JOIN Suppliers S ON P.supplier_id = S.supplier_id GROUP BY S name	0 row(s) returned	0.016 sec / 0.000 sec
37	20:31:09	SELECT type FROM Medicines GROUP BY type HAVING AVG(unit_price) > 100.00 LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec

JOINS Practice Questions

13. List all purchase records along with medicine and supplier names.

SELECT

P.purchase_id,

M.name AS medicine_name,

S.name AS supplier_name,

P.quantity,

P.purchase_date

FROM Purchases P

JOIN Medicines M ON P.medicine_id = M.medicine_id

JOIN Suppliers S ON P.supplier_id = S.supplier_id;

The screenshot displays the MySQL Workbench interface. The SQL Editor window contains the following query:

```
-- JOINS Practice Questions
-- 13. List all purchase records along with medicine and supplier names.
SELECT
  P.purchase_id,
  M.name AS medicine_name,
  S.name AS supplier_name,
  P.quantity,
  P.purchase_date
FROM Purchases P
JOIN Medicines M ON P.medicine_id = M.medicine_id
JOIN Suppliers S ON P.supplier_id = S.supplier_id;
```

The Results Grid shows the output of the query:

purchase_id	medicine_name	supplier_name	quantity	purchase_date
1	Paracetamol	Health Corp	300	2024-01-05
3	Amoxicillin	Health Corp	200	2024-02-01

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
35	20:29:19	SELECT name FROM Medicines WHERE medicine_id NOT IN (SELECT DISTINCT medicine_id FROM Sa...	3 row(s) returned	0.016 sec / 0.000 sec
36	20:30:08	SELECT S.name FROM Purchases P JOIN Suppliers S ON P.supplier_id = S.supplier_id GROUP BY S.name ...	0 row(s) returned	0.016 sec / 0.000 sec
37	20:31:09	SELECT type FROM Medicines GROUP BY type HAVING AVG(unit_price) > 100.00 LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
38	20:32:44	SELECT P.purchase_id, M.name AS medicine_name, S.name AS supplier_name, P.quantity, P...	2 row(s) returned	0.000 sec / 0.000 sec

14. Show all sales with corresponding medicine name and type.

SELECT

S.sale_id,

M.name AS medicine_name,

M.type,

S.quantity,

S.sale_date,

S.total_price

FROM Sales S

JOIN Medicines M ON S.medicine_id = M.medicine_id;

The screenshot displays the MySQL Workbench interface. The SQL Editor contains the following query:

```
-- 14. Show all sales with corresponding medicine name and type.
SELECT
  S.sale_id,
  M.name AS medicine_name,
  M.type,
  S.quantity,
  S.sale_date,
  S.total_price
FROM Sales S
JOIN Medicines M ON S.medicine_id = M.medicine_id;
```

The Results window shows the following data:

sale_id	medicine_name	type	quantity	sale_date	total_price
1001	Paracetamol	Tablet	10	2024-01-15	50.00
1002	Paracetamol	Tablet	20	2024-02-10	100.00
1004	Amoxicillin	Capsule	15	2024-02-20	187.50
1005	Amoxicillin	Capsule	5	2024-03-25	62.50
1006	Cough Syrup	Syrup	20	2024-04-01	1600.00

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
36	20:30:08	SELECT S.name FROM Purchases P JOIN Suppliers S ON P.supplier_id = S.supplier_id GROUP BY S.name	0 row(s) returned	0.016 sec / 0.000 sec
37	20:31:09	SELECT type FROM Medicines GROUP BY type HAVING AVG(unit_price) > 100.00 LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
38	20:32:44	SELECT P.purchase_id, M.name AS medicine_name, S.name AS supplier_name, P.quantity, P...	2 row(s) returned	0.000 sec / 0.000 sec
39	20:33:38	SELECT S.sale_id, M.name AS medicine_name, M.type, S.quantity, S.sale_date, S.total_pri...	5 row(s) returned	0.016 sec / 0.000 sec

DATE, STRING, and COMPLEX Queries

SQL

DATE FUNCTIONS

15. List medicines that will expire within the next 90 days.

SELECT name, expiry_date

FROM Medicines

WHERE expiry_date BETWEEN '2024-03-15' AND DATE_ADD('2024-03-15', INTERVAL 90 days);

-- Note: Replace '2024-03-15' with your database's current date function (e.g., NOW(), GETDATE())

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
183 -- DATE FUNCTIONS
184 -- 15. List medicines that will expire within the next 90 days.
185
186
187 SELECT
188     name,
189     expiry_date
190 FROM
191     Medicines
192 WHERE
193     expiry_date BETWEEN '2024-03-15' AND DATE_ADD('2024-03-15', INTERVAL 90 DAY);
```

The Results window shows the following data:

name	expiry_date
Amoxicillin	2024-06-01
Cough Syrup	2024-05-01

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
38	20:32:44	SELECT P.purchase_id, M.name AS medicine_name, S.name AS supplier_name, P.quantity, P...	2 row(s) returned	0.000 sec / 0.000 sec
39	20:33:38	SELECT S.sale_id, M.name AS medicine_name, M.type, S.quantity, S.sale_date, S.total_gst...	5 row(s) returned	0.016 sec / 0.000 sec
40	20:34:55	SELECT name, expiry_date FROM Medicines WHERE expiry_date BETWEEN 2024-03-15 AND DATE(2024...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL se...	0.000 sec
41	20:36:53	SELECT name, expiry_date FROM Medicines WHERE expiry_date BETWEEN '2024-03-15' AN...	2 row(s) returned	0.016 sec / 0.000 sec

16. Count number of sales made in the last 30 days.

```
SELECT COUNT(sale_id) AS sales_in_last_30_days
```

```
FROM Sales
```

```
WHERE sale_date >= DATE('2024-02-15');
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
188 name,  
189 expiry_date  
190 FROM  
191 Medicines  
192 WHERE  
193 expiry_date BETWEEN '2024-03-15' AND DATE_ADD('2024-03-15', INTERVAL 90 DAY);  
194  
195 -- 16. Count number of sales made in the last 30 days.  
196 SELECT COUNT(sale_id) AS sales_in_last_30_days  
197 FROM Sales  
198 WHERE sale_date >= DATE('2024-02-15');
```

The Results window shows the following output:

sale_in_last_30_days
3

The Output window shows the following log:

#	Time	Action	Message	Duration / Fetch
39	20:33:38	SELECT	S.sale_id, M.name AS medicine_name, M.type, S.quantity, S.sale_date, S.total_pri...	5 row(s) returned 0.016 sec / 0.000 sec
40	20:34:55	SELECT	name, expiry_date FROM Medicines WHERE expiry_date BETWEEN 2024-03-15 AND DATE(2024...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL se... 0.000 sec
41	20:36:53	SELECT	name, expiry_date FROM Medicines WHERE expiry_date BETWEEN 2024-03-15 AN...	2 row(s) returned 0.016 sec / 0.000 sec
42	20:39:57	SELECT	COUNT(sale_id) AS sales_in_last_30_days FROM Sales WHERE sale_date >= DATE(2024-02-15)...	1 row(s) returned 0.000 sec / 0.000 sec

STRING FUNCTIONS

17. List supplier names that start with the letter 'S'.

SELECT name

FROM Suppliers

WHERE name LIKE 'S%';

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
194
195 -- 16. Count number of sales made in the last 30 days.
196 • SELECT COUNT(sale_id) AS sales_in_last_30_days
197 FROM Sales
198 WHERE sale_date >= DATE('2024-02-15');
199
200 -- STRING FUNCTIONS
201 -- 17. List supplier names that start with the letter 'S'.
202 • SELECT name
203 FROM Suppliers
204 WHERE name LIKE 'S%';
```

The Results Grid shows the following data:

name
Swift Supplies

The Output tab at the bottom shows the execution log:

Time	Action	Message	Duration / Fetch
40 20:34:55	SELECT name, expiry_date FROM Medicines WHERE expiry_date BETWEEN 2024-03-15 AND DATE(2024...	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL se...	0.000 sec
41 20:36:53	SELECT name, expiry_date FROM Medicines WHERE expiry_date BETWEEN 2024-03-15 AND...	2 row(s) returned	0.016 sec / 0.000 sec
42 20:39:57	SELECT COUNT(sale_id) AS sales_in_last_30_days FROM Sales WHERE sale_date >= DATE(2024-02-15) ...	1 row(s) returned	0.000 sec / 0.000 sec
43 20:42:01	SELECT name FROM Suppliers WHERE name LIKE 'S%' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

18. Convert all medicine names to uppercase.

SELECT UPPER(name) AS uppercase_name, type
FROM Medicines;

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
198 WHERE sale_date >= DATE('2024-02-15');
199
200 -- STRING FUNCTIONS
201 -- 17. List supplier names that start with the letter 'S'.
202 * SELECT name
203 FROM Suppliers
204 WHERE name LIKE 'S%';
205
206 -- 18. Convert all medicine names to uppercase.
207 * SELECT UPPER(name) AS uppercase_name, type
208 FROM Medicines;
```

The Results window displays the output of the query:

uppercase_name	type
PARACETAMOL	Tablet
AMOXICILLIN	Capsule
VITAMIN C	Tablet
COUGH SYRUP	Syrup
IBUPROFEN	Tablet
DOLO 650	Tablet

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
41	20:36:53	SELECT name, expiry_date FROM Medicines WHERE expiry_date BETWEEN '2024-03-15' AN...	2 row(s) returned	0.016 sec / 0.000 sec
42	20:39:57	SELECT COUNT(sale_id) AS sales_in_last_30_days FROM Sales WHERE sale_date >= DATE('2024-02-15') ...	1 row(s) returned	0.000 sec / 0.000 sec
43	20:42:01	SELECT name FROM Suppliers WHERE name LIKE 'S%' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
44	20:42:35	SELECT UPPER(name) AS uppercase_name, type FROM Medicines LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec

COMPLEX QUERIES

19. Show the top 3 best-selling medicines by quantity.

```
SELECT M.name, SUM(S.quantity) AS total_units_sold
```

```
FROM Sales S
```

```
JOIN Medicines M ON S.medicine_id = M.medicine_id
```

```
GROUP BY M.name
```

```
ORDER BY total_units_sold DESC
```

```
LIMIT 3;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
208 FROM Medicines;
209
210 -- COMPLEX QUERIES
211 -- 19. Show the top 3 best-selling medicines by quantity.
212 SELECT M.name, SUM(S.quantity) AS total_units_sold
213 FROM Sales S
214 JOIN Medicines M ON S.medicine_id = M.medicine_id
215 GROUP BY M.name
216 ORDER BY total_units_sold DESC
217 LIMIT 3;
218
```

The Results window displays the following data:

name	total_units_sold
Paracetamol	30
Amoxicillin	20
Cough Syrup	20

The Output window shows the execution log:

#	Time	Action	Message	Duration / Fetch
42	20:39:57	SELECT COUNT(sale_id) AS sales_in_last_30_days FROM Sales WHERE sale_date >= DATE(2024-02-15) ...	1 row(s) returned	0.000 sec / 0.000 sec
43	20:42:01	SELECT name FROM Suppliers WHERE name LIKE 'S%'; LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
44	20:42:35	SELECT UPPER(name) AS uppercase_name, type FROM Medicines LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
45	20:43:08	SELECT M.name, SUM(S.quantity) AS total_units_sold FROM Sales S JOIN Medicines M ON S.medicine_id = ...	3 row(s) returned	0.000 sec / 0.000 sec

20. List medicines that were purchased but never sold.

```
SELECT M.name  
FROM Medicines M  
JOIN Purchases P ON M.medicine_id = P.medicine_id  
LEFT JOIN Sales S ON M.medicine_id = S.medicine_id  
WHERE S.sale_id IS NULL  
GROUP BY M.name;
```

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
216 ORDER BY total_units_sold DESC  
217 LIMIT 3;  
218  
219 -- 20. List medicines that were purchased but never sold.  
220 *  
221 SELECT M.name  
222 FROM Medicines M  
223 JOIN Purchases P ON M.medicine_id = P.medicine_id  
224 LEFT JOIN Sales S ON M.medicine_id = S.medicine_id  
225 WHERE S.sale_id IS NULL  
226 GROUP BY M.name;
```

The Results window shows the output of the query, which is a single column named 'name'. The Output window shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
43	20:42:01	SELECT name FROM Suppliers WHERE name LIKE 'S%': LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
44	20:42:35	SELECT UPPER(name) AS uppercase_name, type FROM Medicines LIMIT 0, 1000	6 row(s) returned	0.000 sec / 0.000 sec
45	20:43:08	SELECT M.name, SUM(S.quantity) AS total_units_sold FROM Sales S JOIN Medicines M ON S.medicine_id = ...	3 row(s) returned	0.000 sec / 0.000 sec
46	20:43:43	SELECT M.name FROM Medicines M JOIN Purchases P ON M.medicine_id = P.medicine_id LEFT JOIN Sales ...	0 row(s) returned	0.000 sec / 0.000 sec